

EXPERIMENT 14:ALPHA BETA PRUNING

ALGORITHM FOR GAMING

Aim

To implement the **Alpha-Beta Pruning algorithm** for game playing using the **Minimax strategy**, and determine the optimal move by eliminating branches that cannot affect the final decision.

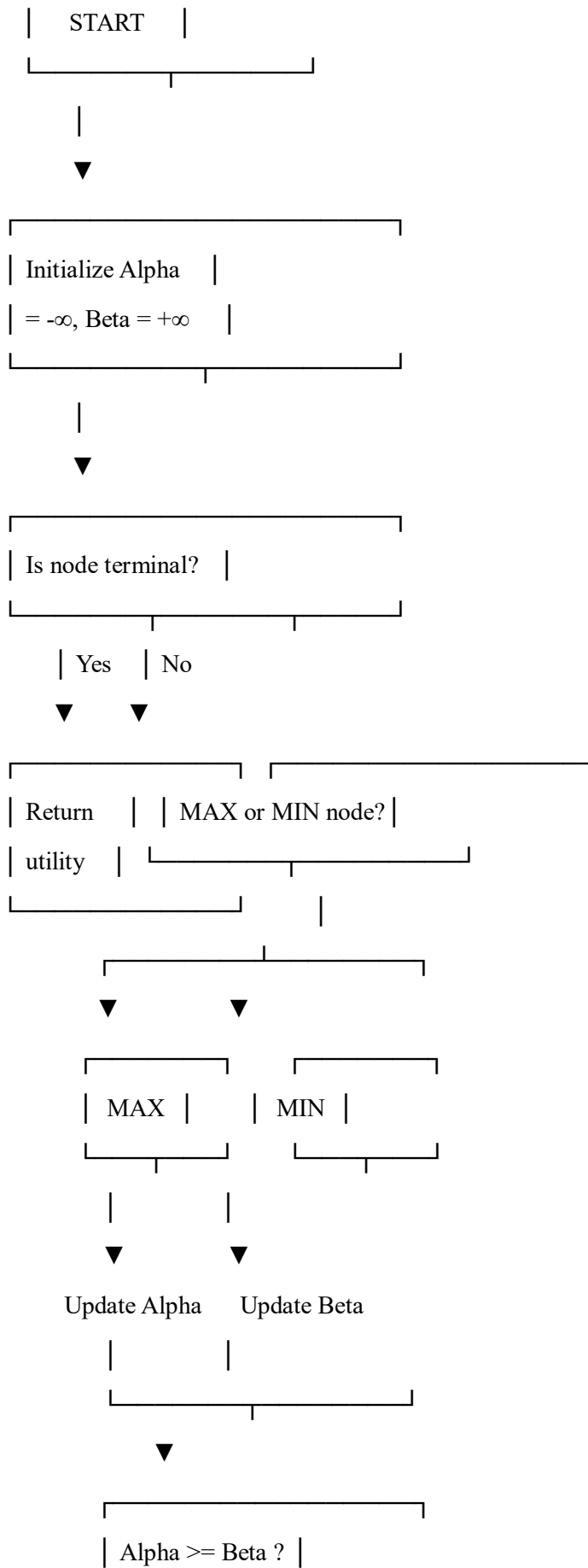
Objective

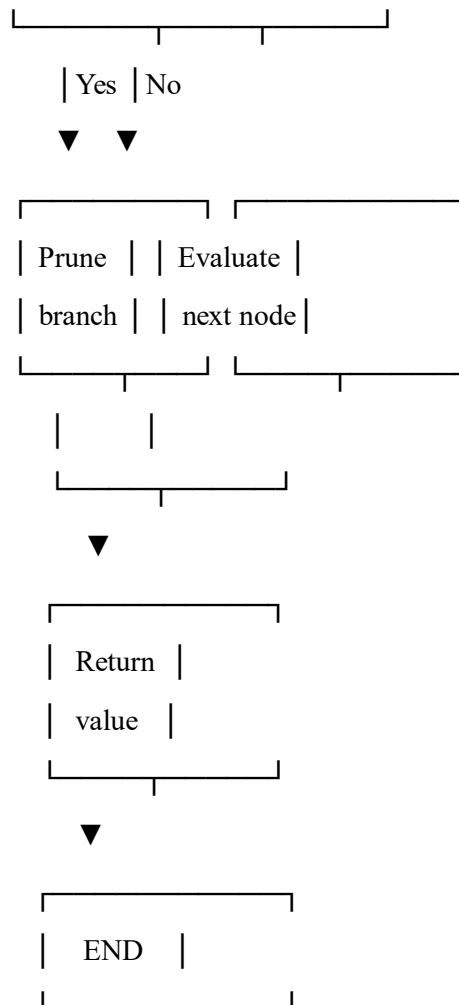
- To understand the Minimax game-playing technique.
- To improve Minimax using Alpha-Beta pruning.
- To reduce the number of nodes evaluated in a game tree.
- To select the best possible move for the maximizing player.

Algorithm

1. Start with the root node of the game tree.
2. Initialize:
 - $\alpha = -\infty$ — the best value guaranteed for MAX.
 - $\beta = +\infty$ — the best value guaranteed for MIN.
3. If the current node is a terminal node, return its utility value.
4. If the current player is **MAX**:
 - Set value = $-\infty$.
 - Evaluate each child recursively.
 - Update value = $\max(\text{value}, \text{child_value})$.
 - Update $\alpha = \max(\alpha, \text{value})$.
 - If $\alpha \geq \beta$, prune the remaining children.
5. If the current player is **MIN**:
 - Set value = $+\infty$.
 - Evaluate each child recursively.
 - Update value = $\min(\text{value}, \text{child_value})$.
 - Update $\beta = \min(\beta, \text{value})$.
 - If $\alpha \geq \beta$, prune the remaining children.
6. Return the calculated value.
7. The move with the highest value is selected by MAX.

Flowchart





Code — Python

Alpha-Beta Pruning for Game Playing

```
def alpha_beta(depth, node_index, maximizing_player,
               values, alpha, beta, height):
```

```
    # Terminal node
```

```
    if depth == height:
```

```
        return values[node_index]
```

```
    if maximizing_player:
```

```
        best = float('-inf')
```

```
        for i in range(2):
```

```

value = alpha_beta(
    depth + 1,
    node_index * 2 + i,
    False,
    values,
    alpha,
    beta,
    height
)

best = max(best, value)
alpha = max(alpha, best)

# Alpha-Beta pruning
if beta <= alpha:
    break

return best

else:
    best = float('inf')

for i in range(2):
    value = alpha_beta(
        depth + 1,
        node_index * 2 + i,
        True,
        values,
        alpha,
        beta,
        height
    )

```

```

        best = min(best, value)
        beta = min(beta, best)

    # Alpha-Beta pruning
    if beta <= alpha:
        break

    return best

# Leaf-node values of the game tree
values = [3, 5, 6, 9, 1, 2, 0, -1]

# Height of the tree
height = 3

# Initial alpha and beta
alpha = float('-inf')
beta = float('inf')

# Find the optimal value
result = alpha_beta(
    0, 0, True, values, alpha, beta, height
)

print("Optimal value:", result)

```

Output:

```

56     return best
57
58
59 # Leaf node values of the game tree
60 values = [3, 5, 6, 9, 1, 2, 0, -1]
61
62 # Height of the game tree
63 height = 3
64
65 # Initial Alpha and Beta values
66 alpha = float('-inf')
67 beta = float('inf')
68
69 # Find the optimal value
70 optimal_value = alpha_beta(
71     0, 0, True, values, alpha, beta, height
72 )
73
74 print("Leaf node values:", values)
75 print("Optimal value:", optimal_value)
76

```

```

Leaf node values: [3, 5, 6, 9, 1, 2, 0, -1]
Optimal value: 5

```

Result

For the given game tree:

Leaf values = [3, 5, 6, 9, 1, 2, 0, -1]

The program produces:

Optimal value: 5

Thus, the **optimal game value is 5**. During the search, Alpha-Beta pruning eliminates branches that do not need to be evaluated.

Conclusion

The **Alpha-Beta Pruning algorithm** successfully finds the optimal move in a two-player game by improving the Minimax algorithm. It gives the same result as Minimax while evaluating fewer nodes. Therefore, Alpha-Beta pruning makes game-playing algorithms **faster and more efficient**, especially when the game tree is large.